FEDERATED MULTI-AGENT REINFORCEMENT LEARNING FOR PRIVACY-PRESERVING DISTRIBUTED WIRELESS

Project Report

Dataset: augmented_dataset_ready.csv
Dataset size: 15,760 records and 8 columns

Abstract

Wireless networks must allocate limited bandwidth to many users and applications while maintaining low latency and reliable connectivity. Centralised resource allocation can create a single point of data concentration and may respond poorly to changing channel conditions. This project studies a privacy-preserving distributed approach based on Federated Multi-Agent Reinforcement Learning (Fed-MARL). Each agent can learn local resource-allocation decisions from observations such as signal strength, latency, required bandwidth and allocated bandwidth. Instead of sharing raw user-level records, participating agents can share model updates with a coordinating server, subject to an appropriate federated-learning protocol.

The supplied CSV contains 15,760 rows and eight columns covering timestamps, user identifiers, application type, signal strength, latency, required bandwidth, allocated bandwidth and resource allocation. It is suitable for exploratory analysis and for building a simulation of resource allocation. The CSV alone does not demonstrate that a federated or reinforcement-learning model has been trained; the report therefore defines a reproducible methodology and evaluation plan rather than claiming unrun experimental results.

Keywords: Federated learning, multi-agent reinforcement learning, wireless networks, bandwidth allocation, latency, privacy.

1. Introduction

Wireless services such as video, voice, gaming and sensor communication have different quality-of-service requirements. A network controller must balance competing demands while avoiding congestion and wasted capacity. Reinforcement learning offers a way to learn allocation policies through feedback, while multi-agent learning allows multiple decision-makers to act on local observations. Federated learning can coordinate model training across distributed agents without routinely collecting their raw records in one place.

The goal is to design and evaluate a framework in which wireless resource decisions are learned locally, coordinated through federated model aggregation, and compared with simpler baselines.

2. Problem Statement and Objectives

Problem statement. A distributed wireless network must allocate bandwidth to multiple users with changing signal quality and traffic requirements. A centralised approach may require collecting detailed records in one location, while independent agents may learn inconsistent policies. The project investigates whether federated coordination of multi-agent policies can improve allocation performance while reducing raw-data sharing.

Objectives

• Inspect and clean the supplied wireless dataset, checking missing values, duplicates, units and category meanings.

• Define each agent's observation, action space and reward function for a simulated resource-allocation task.

• Train a local reinforcement-learning policy for each agent and aggregate model updates using a federated-learning process.

• Compare Fed-MARL with rule-based allocation, independent agents without federation, and a centralised training baseline where feasible.

• Measure latency, bandwidth satisfaction, allocation efficiency, reward, fairness and training stability.

• Discuss privacy limits honestly: federated learning reduces raw-data movement but does not by itself guarantee privacy.

3. Dataset Description

The uploaded file contains 15,760 rows and 8 columns. The fields indicate a time-stamped record of wireless users and resource allocation. Before training, confirm the dataset's provenance, units and how each value was generated; the column names alone do not establish whether values are measured, simulated or synthetically augmented.

Column	Role in experiment
Timestamp	Time index; parse to datetime and preserve temporal ordering.
User_ID	User identifier; exclude as a numeric feature unless used for grouping.
Application_Type	Application class; decode category mapping before interpreting.
Signal_Strength	Radio/channel condition feature; verify dBm or other unit.
Latency	Observed or generated delay; verify unit and measurement method.
Required_Bandwidth	Traffic demand; verify scale and unit.
Allocated_Bandwidth	Allocation outcome or state; verify unit and meaning.
Resource_Allocation	Possible allocation target/score; confirm its definition.

Initial data-quality checks

Missing cells detected: 0. Exact duplicate rows: 0. These are initial checks only; range validation, timestamp continuity, category balance, outlier inspection and leakage checks should also be performed.

4. Proposed System and Methodology

The proposed workflow has six stages: data preparation, environment construction, local agent learning, federated aggregation, evaluation and reporting. The dataset supplies example states and possible initial conditions, but reinforcement learning requires an environment that returns a reward after each action. If the CSV is only a static table, create a simulator or clearly define a contextual-bandit approximation; do not label ordinary supervised prediction as reinforcement learning.

System flow

Stage	Description
1. Data preparation	Parse timestamps, handle missing values, inspect ranges and split by time to reduce leakage.
2. Environment	Represent each user or base-station controller as an agent; model available capacity and demand.
3. Local policy learning	Each agent observes signal, demand, latency and current allocation, then chooses an allocation action.
4. Reward calculation	Reward meeting demand and low latency; penalise congestion, unmet demand and unfairness.
5. Federated aggregation	Send model parameters/gradients, not raw records, to aggregate compatible local models.
6. Evaluation	Repeat across seeds and compare baselines using the same scenarios.

Example reward design

A starting reward can combine normalised quality-of-service terms: R = w■ × bandwidth satisfaction – w■ × latency penalty – w■ × congestion penalty – w■ × unfairness penalty. Weights should be selected and justified experimentally. Keep units normalised so a large numeric bandwidth value does not overwhelm latency or fairness. A penalty is negative feedback for an action; it does not automatically restart or erase the model. Learning continues and updates policy parameters unless the training code explicitly resets them.

Federated training

At each communication round, participating agents train locally for a fixed number of steps. The coordinator aggregates compatible model updates, for example using a sample-count-weighted average, and redistributes the global parameters. Record communication rounds, local training time, convergence and any clients that fail to participate. Agents must use compatible model architectures and parameter formats.

Privacy and security considerations

Raw records should remain at their local agents when feasible. Model updates can still leak information, so stronger protection may require secure aggregation, differential privacy, access controls and threat-model analysis. The report should state which protections are actually implemented rather than treating federation alone as a privacy guarantee.

5. Implementation and Experimental Plan

Suggested software

• Python 3.10+ with pandas and NumPy for data preparation and analysis.

• Matplotlib or Seaborn for plots; scikit-learn for preprocessing and baseline models where appropriate.

• PyTorch or TensorFlow for policy networks; implement a simple simulator before adding distributed training.

• Flower or another federated-learning framework if an established client/server implementation is preferred.

• Jupyter Notebook or VS Code for reproducible experiments, configuration files and saved metrics.

State, action and reward design

Element	Suggested definition
State	Normalised signal strength, latency, required bandwidth, allocated bandwidth, remaining capacity and application class.
Action	Select an allocation level or a bounded bandwidth adjustment; enforce total-capacity constraints.
Reward	Reward satisfied demand and low latency; penalise overload, unmet demand and unfair allocation.
Episode	A time window or simulated sequence of user requests; document reset rules.
Agent	A user, cell, access point or regional controller. Choose one definition and keep it consistent.

Experiment protocol

• Clean the data and document all transformations; fit scaling parameters on training data only.

• Split by time, not random rows alone, if records are time-dependent. Avoid using future outcomes as input features.

• Build a simulator that applies actions and computes next state and reward; validate capacity constraints.

• Train at least three approaches: rule-based heuristic, independent MARL without federation, and Fed-MARL. Add centralised learning as a reference if possible.

• Run multiple random seeds and identical test scenarios; report mean and standard deviation.

• Save configuration, training rewards, evaluation metrics, checkpoints and software versions.

Evaluation metrics

Metric	Interpretation
Mean latency	Lower is generally better, provided demand remains served.
Bandwidth satisfaction	Share of requested bandwidth delivered, with a clearly defined formula.
Resource utilisation	Allocated capacity divided by available capacity; high utilisation is not automatically better if congestion rises.
Cumulative reward	Tracks learning progress but depends on reward design.
Fairness	For example, Jain's fairness index across users' delivered-demand ratios.
Federated communication cost	Bytes or messages exchanged per round and over full training.
Privacy reporting	State whether secure aggregation or differential privacy is used; do not infer privacy from accuracy.

6. Dataset Summary, Expected Outcomes and Conclusion

Descriptive statistics from the supplied CSV

Variable	Mean	Std. dev.	Minimum	Maximum
Signal_Strength	-81.56	18.85	-124.66	-37.43
Latency	37.72	25.43	-2.83	111.25
Required_Bandwidth	2423.24	3428.40	0.00	14501.25
Allocated_Bandwidth	2672.99	3831.15	0.00	15801.52
Resource_Allocation	74.41	8.45	47.51	93.80

Application_Type frequency

Application_Type	Record count	Percentage
0	1,454	9.2%
1	1,454	9.2%
2	1,454	9.2%
3	1,349	8.6%
4	1,454	9.2%
5	1,454	9.2%
6	1,454	9.2%
7	1,454	9.2%
8	1,325	8.4%
9	1,454	9.2%
10	1,454	9.2%

Expected outcomes (to be verified experimentally)

• A functioning prototype that trains local agents and aggregates their model updates.

• Learning curves showing episode reward and evaluation performance over time.

• Comparison plots for latency, bandwidth satisfaction, fairness and resource utilisation across baselines.

• Evidence of the trade-off between model quality, communication overhead and privacy controls.

• A reproducible explanation of failures, limitations and parameter choices.

Important result note. The values above are descriptive statistics of the uploaded dataset, not performance results from a trained Fed-MARL model. Do not claim improvement percentages until the baselines and proposed model have been run on the same test conditions.

7. Conclusion

This project proposes a federated multi-agent reinforcement-learning approach to wireless resource allocation. The provided dataset offers time-stamped user, channel, demand and allocation fields that can support exploratory analysis and help initialise a simulation. The central experimental task is to turn the table into a well-defined sequential decision environment, train local policies, aggregate updates, and compare against meaningful baselines. Evaluation should balance low latency and demand satisfaction with fairness, capacity constraints, communication cost and privacy risk.

8. References

1. McMahan, B. et al. (2017). Communication-Efficient Learning of Deep Networks from Decentralized Data. AISTATS.
2. Sutton, R. S. and Barto, A. G. (2018). Reinforcement Learning: An Introduction (2nd ed.). MIT Press.
3. Lowe, R. et al. (2017). Multi-Agent Actor-Critic for Mixed Cooperative-Competitive Environments. NeurIPS.
4. Bonawitz, K. et al. (2017). Practical Secure Aggregation for Privacy-Preserving Machine Learning. ACM CCS.

Prepared from the uploaded CSV dataset. Confirm dataset definitions and run the experiments before submitting this as a final empirical report.